

Data Chunking Strategies and Their Impact on RAG Performance

Burak Demirci

Artificial Intelligence Engineering

Bahcesehir University

İstanbul, Turkey

burak.demirci2@bahcesehir.edu.tr

Abstract—This paper compares several data chunking strategies and evaluates their impact on Retrieval-Augmented Generation (RAG) performance. It highlights the strengths and limitations of each method, including semantic, hierarchical, agentic, and multimodal chunking.

Index Terms—Chunking, Retrieval-Augmented Generation (RAG)

I. INTRODUCTION

In many *natural language processing (NLP)* and *information retrieval tasks*, long documents must be divided into smaller meaningful units. This process is called **data chunking**. Chunking operation is a common and critical prerequisite for long texts in models that adapt textual data in order to increase the performance of *retrieval augmented generation (RAG)* models. Using larger chunks of text helps maintain the overall meaning of the context, but also increases the amount of irrelevant or noisy information within each chunk [1].

II. CHUNKING STRATEGIES IN TEXTUAL DATA

To demonstrate chunking strategies, first 3 paragraphs from *The Hobbit* (1937, J. R. R. Tolkien) will be used. First paragraph is given below:

In a hole in the ground there lived a hobbit. Not a nasty, dirty, wet hole, filled with the ends of worms and an oozy smell, nor yet a dry, bare, sandy hole with nothing in it to sit down on or to eat: it was a hobbit-hole, and that means comfort.

Google Colab examples for each chunking strategy can be found at: https://colab.research.google.com/drive/1JmV3i4tKWx_EaXTtqsuxMh-nknnt-r3k?usp=sharing

A. Fixed-Size Chunking

Fixed-size chunking is one of the most straightforward implementations of chunking, where the text is divided into equally sized chunks based on a predefined number of characters.

(Chunk 1) In a hole in the ground there lived a hobbit. Not a nasty, dirty, wet hole, filled with the ends of
(Chunk 2) worms and an oozy smell, nor yet a dry, bare, sandy hole with nothing in it to sit down on or to
(Chunk 3) eat: it was a hobbit-hole, and that means comfort. It had a perfectly round door like a porthole,

The first 3 chunks produced by $n = 100$ character fixed-size chunking.

Its simplicity makes it easy to implement and computationally efficient. However, during the process, sentences could be cut into two different chunks and lose the semantic meaning.

B. Sentence-Based Chunking

Another simple chunking strategy is sentence-based chunking where the text is divided into sentences.

(Chunk 1) In a hole in the ground there lived a hobbit.
(Chunk 2) Not a nasty, dirty, wet hole, filled with the ends of worms and an oozy smell, nor yet a dry, bare, sandy hole with nothing in it to sit down on or to eat: it was a hobbit-hole, and that means comfort.

The first 2 chunks produced by sentence-based chunking.

It can be effective when the sentence length does not exceed the context window of the LLM model. Also, when the given context size is larger than the corresponding paragraph, some sentences can be ignored when retrieving.

C. Sliding Window Chunking

It is an enhanced version of fixed-size chunking strategy. The text is still divided into chunks of equal length; however, each new chunk is generated by sliding a window over the text, causing successive chunks to overlap by a specified number of characters.

(Chunk 1) In a hole in the ground there lived a hobbit. Not a nasty, dirty, wet hole, filled with the ends of
(Chunk 2) ends of worms and an oozy smell, nor yet a dry, bare, sandy hole with nothing in it to sit down on
(Chunk 3) down on or to eat: it was a hobbit-hole, and that means comfort. It had a perfectly round door

The first 3 chunks produced by $w = 100$ character and $n = 10$ on sliding window chunking.

This overlapping helps to decrease the risk of losing contextual information at chunk boundaries.

D. Semantic Chunking

Semantic chunking splits a text into meaningful semantic pieces rather than fixed length or sentences.

(Chunk 1) In a hole in the ground there lived a hobbit. Not a nasty, dirty, wet hole, filled with the ends of worms and an oozy smell, nor yet a dry, bare, sandy hole with nothing in it to sit down on or to eat: it was a hobbit-hole, and that means comfort.
(Chunk 2) It had a perfectly round door like a porthole, painted green, with a shiny yellow brass knob in the exact middle.

The first 2 chunks produced by semantic chunking on percentile threshold (chunk number is set to 10, if not specified chunks are 2).

As a result, irrelevant information will be reduced in the retrieved document on the RAG system, and increases the accuracy of the system.

E. Recursive Chunking

In recursive chunking, text is split into chunks according to user-defined list of separators with priority order. The algorithm first splits with the largest separator, and if a chunk is still too long, it recursively applies smaller separators until the chunk fits within the chunk size limit.

(Chunk 1) In a hole in the ground there lived a
(Chunk 2) hobbit. Not a nasty, dirty, wet hole, filled with the ends of worms and an oozy smell, nor yet a dry, bare, sandy hole with nothing in it to sit down on or to eat: it was a

The first 2 chunks produced by recursive chunking where the separators are ["hobbit", "Baggins"].

So, with this algorithm, the resulting splits depend heavily on the chosen separators. If a frequently occurring separator is placed earlier in the priority list (e.g., "." or ","), the text will be split more often, producing smaller chunks and a larger number of chunks. Conversely, if the early separators are rarely used (e.g., a specific number or an uncommon word), the chunks will be larger and fewer in number. Therefore, this can improve or degrade the accuracy of the RAG system, depending on the separator list.

F. Hierarchical Chunking

Hierarchical chunking is used to split very large and structured documents. The document is split according to its hierarchical order such as sections, subsections, paragraphs or sentences. The splitter identifies the document's structure and creates a multi-level chunk organization.

As a result, using hierarchical chunking on a long PDF, DOCX or HTML files, results with more contextually consistent chunks which improves RAG retrieval accuracy. However, one disadvantage of this strategy is that it requires a well-structured document. If a poorly-structured or unformatted document is given, the parser may fail to correctly identify the hierarchy and produce suboptimal chunks.

G. Agentic Chunking

Agentic chunking implements AI agents to automate split operation. Similar to semantic chunking, agentic chunking also splits using contextual meaning. But it can also adapt based on context, document type, or RAG task.

First, it collects the chunks from other implemented algorithms, then a LLM model labels this chunks and adjusts the boundaries of each chunk to achieve semantical connection in each chunk within the context window.

Due to its dependency on LLM models, this operation can be significantly slower than other chunking methods. However, when applied effectively, it can produce more semantically accurate chunks than traditional approaches.

III. MULTIMODAL CHUNKING

Multimodal chunking is designed to handle multiple data modalities such as images, audio, video, and text. Instead of relying solely on textual chunking strategies, other modalities are also preprocessed and converted before being passed to the RAG pipeline.

To implement multimodal chunking, additional models - such as OCR (Object Character Recognition) models for text detection in image or Speech-to-Text models to recognize audial data- are also implemented alongside an LLM model.

As a result, multimodal chunking allows RAG systems to handle complex real-world documents more effectively by preserving relationships between text and visual or audial elements.

IV. CONCLUSION

TABLE I
COMPARISON OF CHUNKING STRATEGIES

Method	Splitting Logic	Advantages	Disadvantages
Fixed-Size	Fixed number of characters	Simple, predictable	May break semantic meaning
Sliding Window	Fixed size with overlap	Better context retention	Produces more chunks
Sentence-Based	Sentence boundaries	Grammatically coherent chunks	Long sentences may exceed limits
Recursive	Separator-priority splitting	Flexible, adaptive	Quality depends on separator list
Semantic	Embedding-based similarity	Highly meaningful chunks	Computationally expensive
Hierarchical	Document structure	Great for structured docs	Weak on unstructured text
Agentic	LLM-based adaptive splitting	Highest semantic accuracy	Slow and costly

REFERENCES

- [1] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.
- [2] S. Ranganath. (2025, mar) Rag 101: Chunking strategies. [Online]. Available: <https://towardsdatascience.com/rag-101-chunking-strategies-fdc6f6c2aaec/>